Experiment No. 7

Title:

Demonstration of SSL/TLS (TLS 1.3) Protocol Using Wireshark in Kali Linux

Aim:

To demonstrate the working of the SSL/TLS protocol (specifically **TLS 1.3**) by analysing HTTPS traffic of a secure website using Wireshark in Kali Linux.

Objectives:

After completing this experiment, students will be able to:

1. Understand the purpose of SSL/TLS protocol.
2. Identify TLS 1.3 handshake messages.
3. Analyze digital certificates exchanged during TLS handshake.
4. Differentiate between HTTP and HTTPS communication.
5. Observe encrypted communication using network analyzer tools.
6. Relate TLS with digital signatures and authentication mechanisms studied in previous experiments.

Theory :

🔐 **Introduction to SSL/TLS**

SSL (Secure Sockets Layer) was the original security protocol used for securing web communication. However, SSL versions are now deprecated due to vulnerabilities.

Modern systems use:

TLS (Transport Layer Security)

The latest secure version widely used today is:

TLS 1.3 (RFC 8446)

🔐 **Why TLS is Required?**

TLS provides:

1. **Confidentiality** – Data encryption
2. **Integrity** – Prevents tampering
3. **Authentication** – Verifies server identity
4. **Forward Secrecy** – Past sessions remain secure even if key is compromised

---

🔐 TLS 1.3 Features

TLS 1.2	TLS 1.3
Multiple handshake rounds	Single round-trip (1-RTT)
Weak cipher suites supported	Weak ciphers removed
RSA key exchange allowed	Only Forward Secret methods
Slower handshake	Faster and secure

🔐 TLS 1.3 Working (Simplified Flow)

1. TCP 3-Way Handshake
2. Client Hello
3. Server Hello
4. Certificate Exchange
5. Key Exchange
6. Finished Messages
7. Encrypted Application Data

After handshake completion, all communication is encrypted.

🔐 Role of Digital Certificate

The server sends an X.509 certificate that contains:

- Server public key
- Certificate authority (CA) signature
- Validity period
- Issuer information

This connects directly to your previous **Digital Signature experiment**.

Tools Required

- Kali Linux
- Wireshark
- Web browser (Firefox/Chrome)
- Internet connection

Procedure

Step 1: Start Wireshark

Open terminal:

wireshark

Select active network interface (eth0 or wlan0).

Click **Start Capture**.

Step 2: Apply Display Filter

After capture begins, apply filter:

tls

OR

tcp.port == 443

Step 3: Access Secure Website

Open browser and visit:

https://www.google.com

OR

https://example.com

Allow page to load completely.

Step 4: Stop Packet Capture

Return to Wireshark and stop capturing.

Step 5: Analyze Packets

- **Observe TCP Handshake**

Apply filter:

tcp

Identify:

- SYN
- SYN-ACK
- ACK

- **Observe Client Hello**

Filter:

tls.handshake.type == 1

Expand:

Transport Layer Security → Client Hello

Observe:

- Supported TLS versions
- Cipher suites
- Extensions

- **Observe Server Hello**

Filter:

tls.handshake.type == 2

Observe:

- Selected TLS version (TLS 1.3)
- Selected cipher suite

- **Observe Certificate**

Filter:

tls.handshake.certificate

Expand certificate details.

Observe:

- Issuer
- Subject
- Public Key
- Signature Algorithm

🔹 Observe Encrypted Application Data

Filter:

tls.record.content_type == 23

You will see:

Application Data

Payload appears encrypted.

Output :

Students should observe:

1. TCP 3-way handshake packets.
2. TLS 1.3 Client Hello and Server Hello.
3. Digital certificate transmission.
4. Cipher suite negotiation (e.g., TLS_AES_128_GCM_SHA256).
5. Encrypted Application Data packets.
6. No readable HTTP content inside HTTPS session.

Observations :

Observation	Explanation
TCP handshake occurs first	TLS works over TCP
Client Hello sent	Client proposes supported cryptography
Server Hello sent	Server selects cipher
Certificate exchanged	Server authentication

Observation	Explanation
Application Data encrypted	Secure communication established
HTTP content not visible	Encryption ensures confidentiality

Comparative Demonstration (HTTP vs HTTPS)

Now visit:

http://neverssl.com

Apply filter:

http

Observe:

- Plain GET request
- Visible headers
- Visible webpage data

Comparison:

HTTP	HTTPS
Plaintext	Encrypted
Port 80	Port 443
Data visible in Wireshark	Data encrypted

Conclusion :

This experiment successfully demonstrated the working of TLS 1.3 protocol.

It was observed that:

- TLS handshake establishes secure session.
- Server identity is verified using digital certificates.
- Data transmitted after handshake is encrypted.
- HTTPS provides confidentiality, integrity, and authentication.

TLS 1.3 enhances performance and security compared to earlier versions.